

Capítulo 3:

Estructuras de datos lineales y no lineales

Laura Viviana Galvis

Escuela de Ingeniería de Sistemas e Informática
Facultad de Ingenierías Fisicomecánicas
Universidad Industrial de Santander

2025-2

Escuela de
Ingeniería de
Sistemas e
Informática



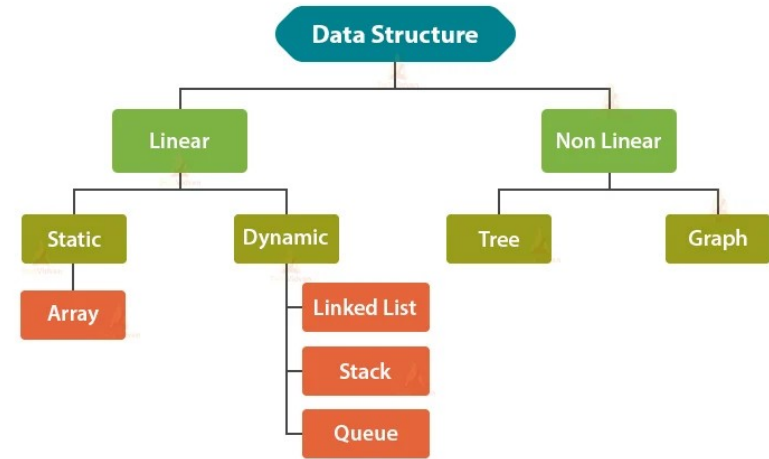
Universidad
Industrial de
Santander



Contenidos del capítulo

1. Estructuras de datos lineales

- a. Arrays
- b. Listas enlazadas
- c. Pilas
- d. Colas

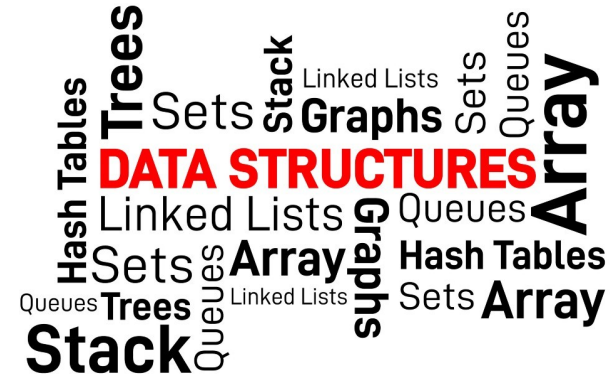


2. Estructuras de datos no lineales

- a. Árboles (binarios y balanceados)
- b. Grafos
- c. Montículo (heap)

3. De acceso directo:

- 4. a. Mapas, diccionarios y tablas hash



1. Estructuras de datos lineales

1.1 Listas Enlazadas

- Estructura de datos que representa una secuencia de **nodos**, conectados a través de **enlaces**
- Cada nodo está dividido en 2 (simple) o 3 partes (doble)
 - Simple: Donde se almacena el dato, y la dirección del siguiente nodo
 - Doble: Donde se almacena el dato, la dirección del siguiente nodo, y la dirección del anterior nodo
- Los enlaces tienen terminación NULL (excepto circulares)
 - Cuando el puntero al siguiente nodo es NULL, la lista ha llegado al final

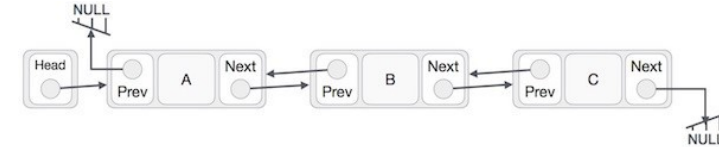
Aplicaciones:

- Gestión de la lista de canciones en reproductor multimedia.
 - Inicio/final, agregar canción, escuchar anterior/siguiente
- Combinaciones de teclado para *shortcuts*
- Historial de navegación
- Operaciones “hacer”, “deshacer”
- Una lista circular se usa en los videojuegos multijugador para controlar el orden de los jugadores.

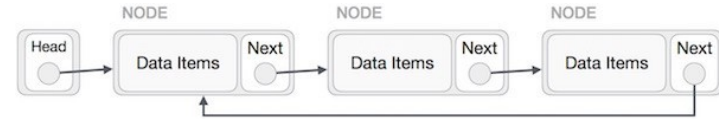
Lista enlazada simple



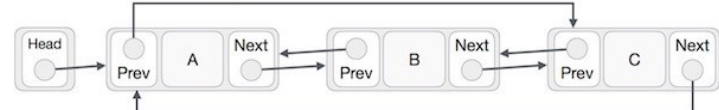
Lista enlazada doble



Lista enlazada circular (no tiene fin)



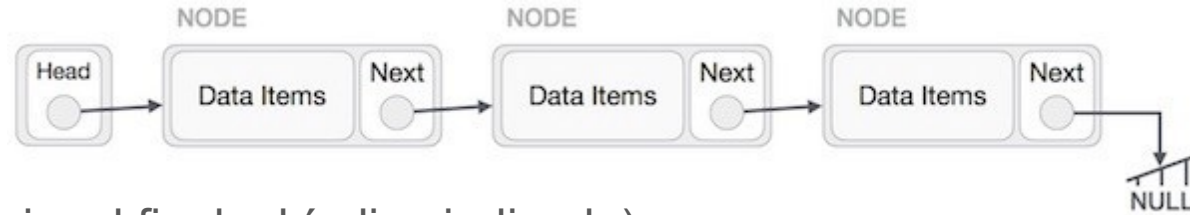
Lista enlazada circular doble



1.1 Lista Enlazada Simple

Estructura: Nodo (con **datos** y el **puntero** al siguiente)

Operaciones permitidas:



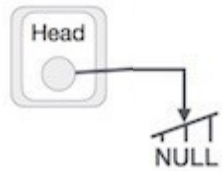
- **Creación**
- **Insertar** elementos (al inicio, al final, al índice indicado)
- **Recorrer** la lista para imprimirla
- Verificar si está **vacía**
- **Buscar** elementos
- **Eliminar** elementos (al inicio, al final, al índice indicado)

Construyendo un Nodo (**DIY**)



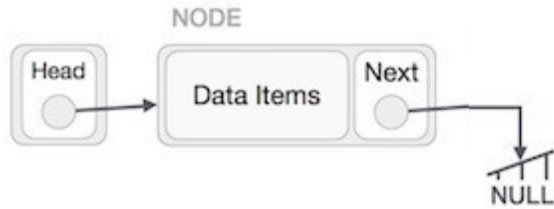
```
1  public class Nodo {
2
3      public int dato;
4      public Nodo siguiente = null;
5
6      public Nodo(int dato){
7          this.dato = dato;
8      }
9
10 }
```

Construyendo una Lista (**DIY**)



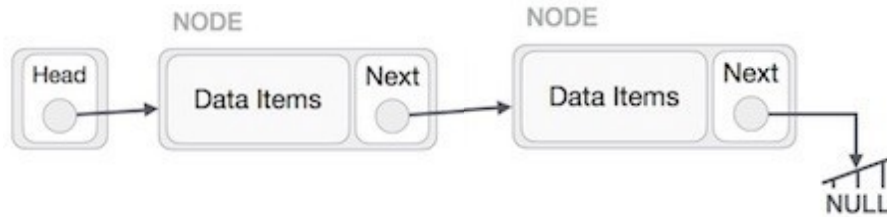
```
1  public class Lista {  
2  
3      private Nodo cabeza;  
4  
5      public Lista(){  
6          this.cabeza = null;  
7      }  
8  }
```

Insertando elementos **al inicio de** una Lista (**DIY**)



```
1  public void insertarNodoInicio(int dato){  
2  
3      Nodo nodoIni = new Nodo(dato);  
4      nodoIni.siguiente = cabeza;  
5      cabeza = nodoIni;  
    }
```

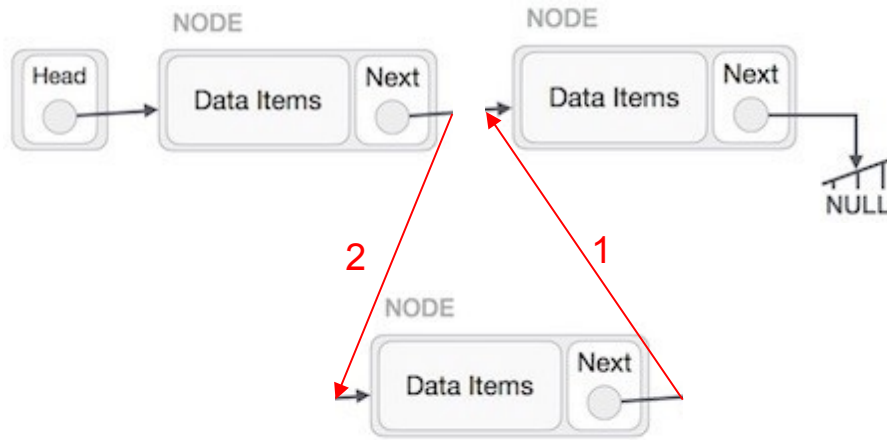

Insertando elementos **al final de** una Lista (**DIY**)



```

1  public void insertarNodoFinal(int dato){
2
3      Nodo nodoFin = new Nodo(dato);
4      Nodo nodoRecorre = cabeza;
5
6      while (nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8      }
9
10     nodoRecorre.siguiente = nodoFin;
11 }
  
```

Insertando elementos al índice indicado de una Lista (DIY)



```

1  public void insertarNodoIndice(int dato, int idx){
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < idx && nodoRecorre.siguiete != null){
7          nodoRecorre = nodoRecorre.siguiete;
8          cont++;
9      }
10
11     nodoIndice.siguiete = nodoRecorre.siguiete;
12     nodoRecorre.siguiete = nodoIndice;
13
14 }
    
```



Imprimiendo elementos de una Lista (**DIY**)

```
1 public void imprimirLista(){
2     Nodo nodoRecorre = cabeza;
3
4     while (nodoRecorre != null){
5         System.out.print(nodoRecorre.dato + " -> ");
6         nodoRecorre = nodoRecorre.siguiente;
7     }
8     System.out.println("NULL");
9 }
```

```
1 run:
2 NULL
3 10 -> NULL
4 5 -> 10 -> NULL
5 1 -> 5 -> 10 -> NULL
6 1 -> 5 -> 10 -> 20 -> NULL
7 1 -> 5 -> 10 -> 20 -> 30 -> NULL
8 1 -> 5 -> 10 -> 15 -> 20 -> 30 -> NULL
9 1 -> 3 -> 5 -> 10 -> 15 -> 20 -> 30 -> NULL
```

Probando...



```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Insertando al inicio");
8      listaEjemplo.insertarNodoInicio(10);
9      listaEjemplo.imprimirLista();
10
11     listaEjemplo.insertarNodoInicio(5);
12     listaEjemplo.imprimirLista();
13
14     listaEjemplo.insertarNodoInicio(1);
15     listaEjemplo.imprimirLista();
16
17     System.out.println();
18     System.out.println("Insertando al final");
19     // Insertando al final
20     listaEjemplo.insertarNodoFinal(20);
21     listaEjemplo.imprimirLista();
22     listaEjemplo.insertarNodoFinal(30);
23     listaEjemplo.imprimirLista();
24
25     System.out.println();
26     System.out.println("Insertando en índice");
27     // Insertando al indice
28     listaEjemplo.insertarNodoIndice(15, 2);
29     listaEjemplo.imprimirLista();
30     listaEjemplo.insertarNodoIndice(3, 4);
31     listaEjemplo.imprimirLista();
32 }
```

¿Cómo estamos contando?

```

1  public void insertarNodoIndice(int dato, int idx){
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < idx && nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     nodoIndice.siguiente = nodoRecorre.siguiente;
11     nodoRecorre.siguiente = nodoIndice;
12 }

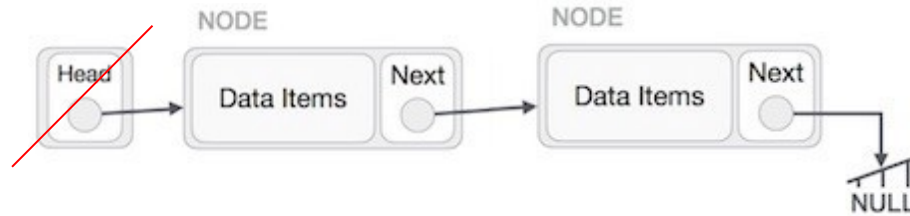
```

```

1  public void insertarNodoIndice(int dato, int idx) {
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < (idx-1) && nodoRecorre.siguiente != null) { // Empezando desde la posición 1
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     if (cont==(idx-1)){
11         nodoIndice.siguiente = nodoRecorre.siguiente; // Primero conectamos el Indice con lo que queda hasta la cola
12         nodoRecorre.siguiente = nodoIndice;          // Despues conectamos lo del inicio con el Indice
13     }
14 }
15 }

```

Eliminando elementos **al inicio** de una Lista (**DIY**)



```

1  public void eliminarNodoInicio() {
2      Nodo inicio = cabeza;
3      cabeza = inicio.siguiete;
4      inicio.siguiete = null; //romper el enlace
5      tamano--;
6  }

```

Opción 2:

```

1  public void eliminarNodoInicio() {
2      cabeza = cabeza.siguiete;
3      tamano--;
4  }

```

Conociendo el tamaño de la lista? (DIY)

```
1  private int tamano;  
2  
3  public int getTamano(){  
4      return tamano;  
5  }  
//NOTA: Se debe agregar los correspondientes tamano++ y tamano--
```

Verificar si la lista está vacía? (DIY)

```
1  public boolean estaVacia(){  
2      if(cabeza == null){  
3          return true;  
4      }  
5      else{  
6          return false;  
7      }  
8  }
```

```

1  public class Lista {
2
3      private Nodo cabeza;
4      Private int tamano;
5
6      public Lista(){
7          this.cabeza = null;
8          this.tamano = 0;
9      }
10 }

```

```

1  public void insertarNodoFinal(int dato){
2
3      Nodo nodoFin = new Nodo(dato);
4      Nodo nodoRecorre = cabeza;
5
6      while (nodoRecorre.siguiente != null){
7          nodoRecorre = nodoRecorre.siguiente;
8      }
9      nodoRecorre.siguiente = nodoFin;
10     tamano++;
11 }

```

```

1  public void insertarNodoIndice(int dato, int idx) {
2      Nodo nodoIndice = new Nodo(dato);
3      Nodo nodoRecorre = cabeza;
4
5      int cont = 0;
6      while (cont < (idx-1) && nodoRecorre.siguiente != null) {
7          nodoRecorre = nodoRecorre.siguiente;
8          cont++;
9      }
10     if (cont==(idx-1)){
11         nodoIndice.siguiente = nodoRecorre.siguiente; // Primero conectamos el Indice con lo que queda hasta la cola
12         nodoRecorre.siguiente = nodoIndice;           // Despues conectamos lo del inicio con el Indice
13         tamano++;
14     }
15 }

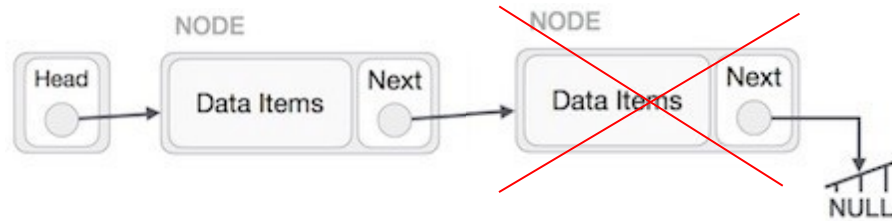
```

```

1  public void insertarNodoInicio(int dato){
2
3      Nodo nodoIni = new Nodo(dato);
4      nodoIni.siguiente = cabeza;
5      cabeza = nodoIni;
6      Tamano++;
7  }

```

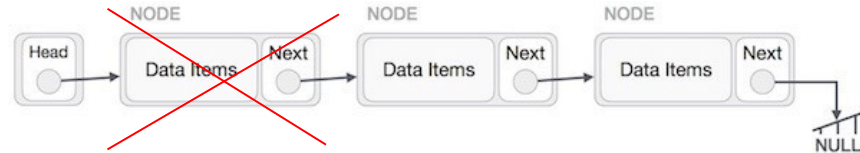

Eliminando elementos **al final** de una Lista (**DIY**)



```

1  public void eliminarNodoFinal() {
2      if (cabeza.siguiete == null) { //Si solo hay uno (corresponde a la cabeza)
3          cabeza = null; // Entonces hacer la lista vacía
4      } else {
5          Nodo nodoRecorre = cabeza;
6
7          while (nodoRecorre.siguiete.siguiete != null) {
8              nodoRecorre = nodoRecorre.siguiete;
9          }
10         nodoRecorre.siguiete = null;
11     }
12     tamano--;
13 }
    
```

Eliminando elementos **al índice indicado** de una Lista (**DIY**)



```

1  public void eliminarNodoIndice(int idx) {
2      Nodo nodoRecorre = cabeza;
3
4      int cont = 0;
5      while (cont < (idx - 1) && nodoRecorre.siguiente.siguiente != null) {
6          nodoRecorre = nodoRecorre.siguiente;
7          cont++;
8      }
9
10     Nodo aux = nodoRecorre.siguiente;           //Nodo a eliminar
11     nodoRecorre.siguiente = aux.siguiente;
12     aux.siguiente = null;                       //romper el enlace
13
14     tamano--;
15 }
  
```

```

1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Eliminando al inicio");
8      // Eliminando al principio
9      listaEjemplo.eliminarNodoInicio();
10     listaEjemplo.imprimirLista();
11     System.out.println("Inicio: Tamaño de la lista es: "+listaEjemplo.getTamano());
12
13     listaEjemplo.eliminarNodoInicio();
14     listaEjemplo.imprimirLista();
15     System.out.println("Inicio: Tamaño de la lista es: "+listaEjemplo.getTamano());
16
17     System.out.println();
18     System.out.println("Eliminando al final");
19     listaEjemplo.eliminarNodoFinal();
20     listaEjemplo.imprimirLista();
21     System.out.println("Tamaño de la lista es: "+listaEjemplo.getTamano());
22
23     System.out.println();
24     System.out.println("Eliminando en índice");
25     listaEjemplo.eliminarNodoIndice(2);
26     listaEjemplo.imprimirLista();
27     System.out.println("Idx: Tamaño de la lista es: "+listaEjemplo.getTamano());
28
29     listaEjemplo.eliminarNodoIndice(1);
30     listaEjemplo.imprimirLista();
31     System.out.println("Idx: Tamaño de la lista es: "+listaEjemplo.getTamano());
32 }
33 }
```

Buscando elementos de una Lista (DIY)

```
1  public int buscarNodo(int idx){  
2      Nodo nodoRecorre = cabeza;  
3  
4      int cont = 0;  
5      while (cont < idx && nodoRecorre.siguiente != null){  
6          nodoRecorre = nodoRecorre.siguiente;  
7          cont++;  
8      }  
9      return nodoRecorre.dato;  
10 }  
11  
12  
13
```

Probando...



```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Buscando Nodo");
8      // Buscar nodo al indice idx
9      int idx = 2;
10     int valDato = listaEjemplo.buscarNodo(idx);
11     System.out.println("Valor del nodo (" + idx + ") = " + valDato);
12     System.out.println("Tamaño de la lista es: " + listaEjemplo.getTamano());
13 }
14 }
```

Buscando elementos de una Lista (**DIY**)

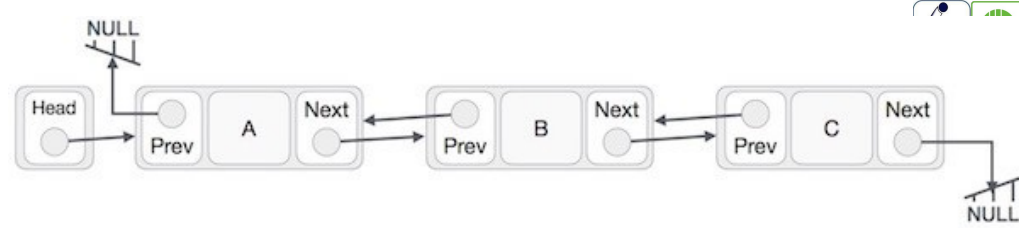
Ejercicio: ¿Cómo hacemos para buscar un “valor” específico y no una posición?

```
1  public int buscarNodoValor(int val) {  
2      Nodo nodoRecorre = cabeza;  
3  
4      int cont = 0;  
5      while (nodoRecorre.dato!=val && nodoRecorre.siguiente != null) {  
6          nodoRecorre = nodoRecorre.siguiente;  
7          cont++;  
8      }  
9      return cont;  
10 }
```

Ejercicio: ¿Cómo hacemos para buscar un “valor” específico y no una posición?

```
1  public static void main(String[] args) {
2      // TODO code application logic here
3      Lista listaEjemplo = new Lista();
4      listaEjemplo.imprimirLista();
5
6      System.out.println();
7      System.out.println("Buscando Nodo con valor");
8      // Buscar nodo con valor val
9      int val = 20;
10     int pos = listaEjemplo.buscarNodoValor(val);
11     System.out.println("Posicion del nodo con valor (" + val + ") = " + pos);
12     System.out.println("Tamaño de la lista es: " + listaEjemplo.getTamano());
13 }
14 }
```

1.2 Lista Enlazada Doble

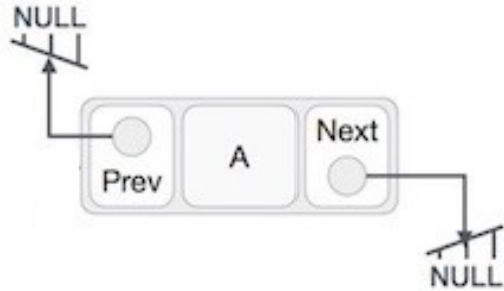


Estructura: Nodo (con **datos** y 1 **puntero** al **siguiente** y 1 puntero al **anterior**)

Operaciones permitidas:

- **Creación**
- **Insertar** elementos (al inicio, al final, al índice indicado)
- **Recorrer** la lista para imprimirla
- Verificar si está **vacía**
- **Buscar** elementos
- **Eliminar** elementos (al inicio, al final, al índice indicado)

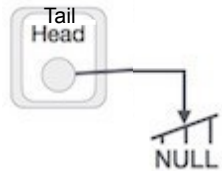
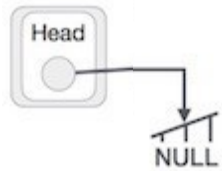
Construyendo un Nodo para Lista doble (DIY)



```

1  public class Nodo2 {
2
3      public int dato;
4      public Nodo2 siguiente = null;
5      public Nodo2 anterior= null;
6
7      public Nodo2(int dato){
8          this.dato = dato;
9      }
10
11 }
    
```

Construyendo una Lista (**DIY**)



```
1  public class Lista2 {  
2  
3      private Nodo2 cabeza, cola;  
4  
5      public Lista2(){  
6          this.cabeza = null;  
7          this.col = null;  
8      }  
9  }
```

Conociendo el tamaño de la Lista? (DIY)

```
1  private int tamano;  
2  
3  public int getTamano(){  
4      return tamano;  
5  }  
//NOTA: Se debe agregar los correspondientes tamano++ y tamano--
```

Verificar si la Lista está vacía? (DIY)

```
1  public boolean estaVacia(){  
2      if(cabeza == null){  
3          return true;  
4      }  
5      else{  
6          return false;  
7      }  
8  }
```

Imprimiendo elementos de una Lista (**DIY**)

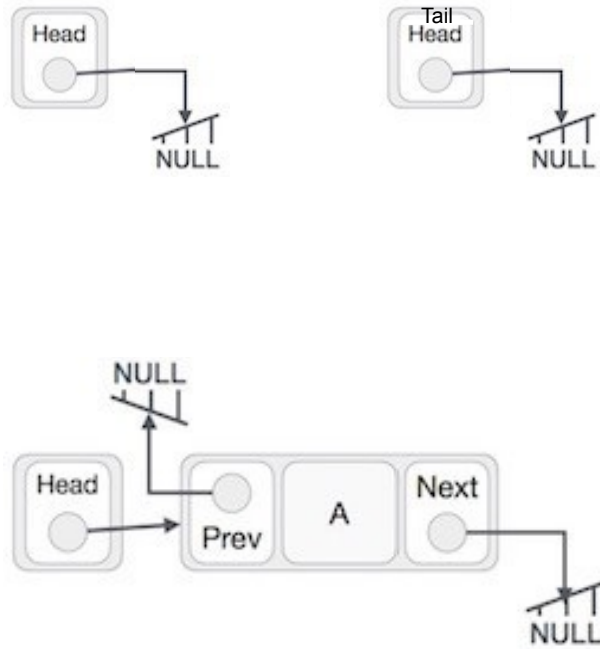
De inicio a fin

```
1  public void imprimirListaIniAFin() {  
2      Nodo2 nodoRecorre = cabeza;  
3  
4      System.out.print("CABEZA -> ");  
5      while (nodoRecorre != null) {  
6          System.out.print(nodoRecorre.dato + " -> ");  
7          nodoRecorre = nodoRecorre.siguiente;  
8      }  
9      System.out.println("FINAL");  
10 }
```

De fin a inicio

```
1  public void imprimirListaFinAIni() {  
2      Nodo2 nodoRecorre = cola;  
3  
4      System.out.print("FINAL -> ");  
5      while (nodoRecorre != null) {  
6          System.out.print(nodoRecorre.dato + " -> ");  
7          nodoRecorre = nodoRecorre.anterior;  
8      }  
9      System.out.println("INICIO");  
10 }
```

Insertando elementos **al inicio** de una Lista (**DIY**)

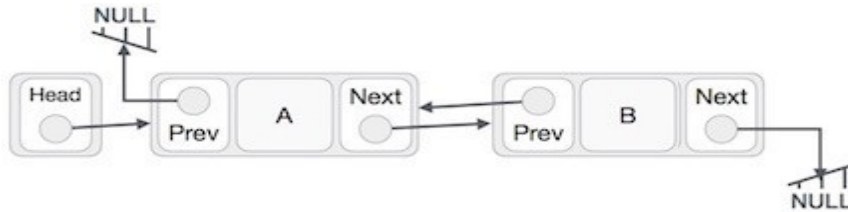


```

1  public void insertarNodoInicio(int dato) {
2      Nodo2 nuevo = new Nodo2(dato);
3
4      if (estaVacia()) {
5          cola = nuevo;
6      } else {
7          cabeza.anterior = nuevo;
8      }
9
10     nuevo.siguiente = cabeza;
11     nuevo.anterior = null;
12
13     cabeza = nuevo;
14     tamano++;
15 }

```

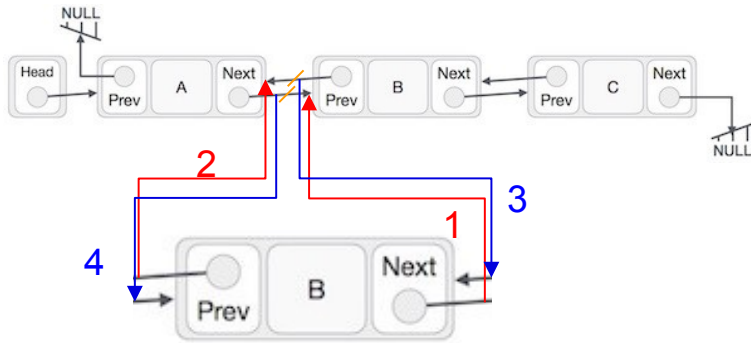
Insertando elementos **al final de** una Lista (**DIY**)



```

1  public void insertarNodoFinal(int dato) {
2      Nodo2 nuevo = new Nodo2(dato);
3
4      if (estaVacia() == true) { // Si está vacía la lista
5          cabeza = nuevo;
6      } else {
7          cola.siguiente = nuevo;
8          nuevo.anterior = cola;
9      }
10
11     cola = nuevo;
12     tamano++;
13 }
  
```

Insertando elementos al índice indicado de una Lista (DIY)



```

1  public void insertarNodoPosicion(int dato, int pos) {
2      Nodo2 nuevo = new Nodo2(dato);
3      Nodo2 recorre;
4
5      if (estaVacia() == true && pos != 0) { // Si está vacía la lista
6          System.out.println("Posicion no existe");
7      } else if (estaVacia() == true && pos == 0) {
8          cabeza = nuevo;
9          cola = nuevo;
10     } else {
11         recorre = cabeza;
12         int cont = 0;
13
14         while (cont < pos-1 && recorre.siguiente != null) {
15             recorre = recorre.siguiente;
16             cont++;
17         }
18
19         if (cont == pos-1) {
20             nuevo.siguiente = recorre.siguiente;
21             nuevo.anterior = recorre;
22
23             recorre.siguiente.anterior = nuevo;
24             recorre.siguiente = nuevo;
25         } else {
26             System.out.println("Posicion no existe");
27         }
28     }
29     tamaño++;
30 }
31 
```

Probando...



```
1 public class Listas_Enlazadas_Dobles {
2     public static void main(String[] args) {
3         // TODO code application logic here
4         Lista2 listaEjemplo = new Lista2();
5         System.out.println("Nueva lista");
6         listaEjemplo.imprimirListaIniAFin();
7         listaEjemplo.imprimirListaFinAIni();
8
9         System.out.println("");
10        System.out.println("Insertando al final");
11        listaEjemplo.insertarNodoFinal(10);
12        listaEjemplo.imprimirListaIniAFin();
13        //listaEjemplo.imprimirListaFinAIni();
14
15        System.out.println("");
16        System.out.println("Insertando al inicio");
17        listaEjemplo.insertarNodoInicio(5);
18        listaEjemplo.imprimirListaIniAFin();
19
20        System.out.println("");
21        System.out.println("Insertando al final");
22        listaEjemplo.insertarNodoFinal(20);
23        listaEjemplo.imprimirListaIniAFin();
24
25        System.out.println("");
26        System.out.println("Insertando al final");
27        listaEjemplo.insertarNodoFinal(30);
28        listaEjemplo.imprimirListaIniAFin();
29
30        System.out.println("");
31        System.out.println("Insertando Nodo en posicion 2");
32        listaEjemplo.insertarNodoPosicion(1, 2);
33        listaEjemplo.imprimirListaIniAFin();
34    }
35 }
```